


Chapter 1 :

How to recognize digits?

0	4	1	9	2	1	3	1	4	3
5	3	6	1	7	2	8	6	9	4
0	9	1	1	2	4	3	2	7	3
8	6	9	0	5	6	0	7	6	1
8	7	9	3	9	8	5	9	3	3
0	7	4	9	8	0	9	4	7	4
4	6	0	4	5	6	1	0	0	1
7	1	6	3	0	2	1	1	7	8
0	2	6	7	8	3	9	0	4	6
7	4	6	8	0	7	8	3	1	5

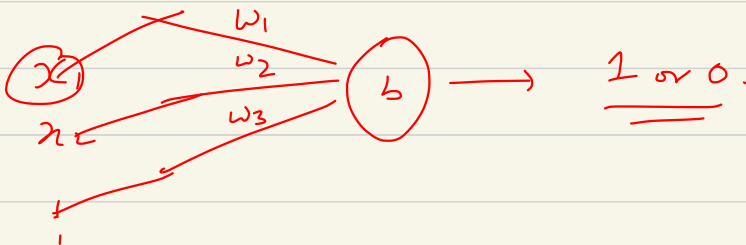
'Simple' NN $\rightarrow$ 96%.

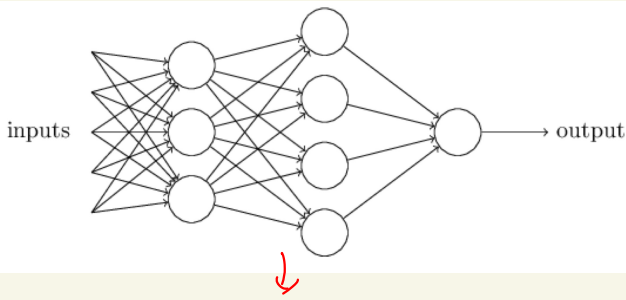
Few more tricks $\rightarrow$ 99%.

Hard to write an explicit program.

Perceptrons

$$1(w^T x + b \geq 0).$$



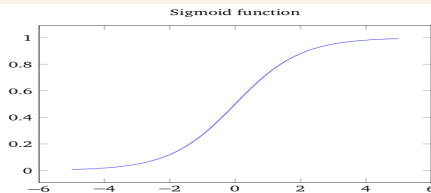


Network of Perceptrons $\rightarrow$ Function Class.

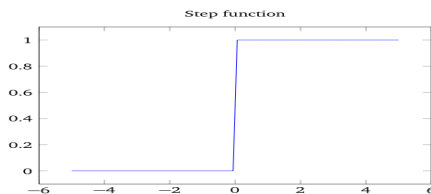
Learning: How to learn?
 Small Change in parameters $\Rightarrow$ Small change in output.

Perceptrons are not continuous.

Use Sigmoid f. $\sigma(t) = \frac{e^t}{1+e^t}$.

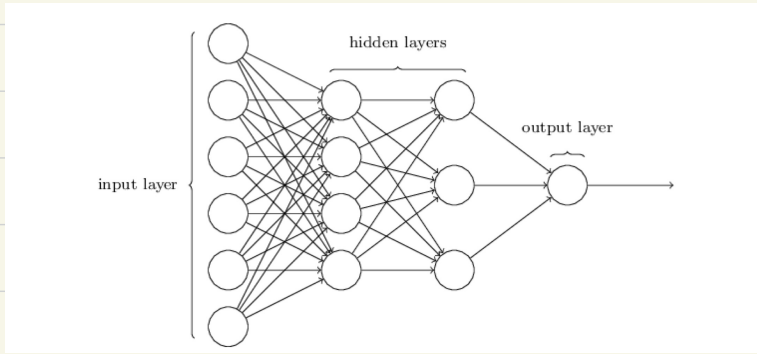


This shape is a smoothed out version of a step function:



Typical Architecture of Feed Forward NN

Matrix
Notation
 W_L
 $= \sigma W_L \sigma W_1(z)$



$$a_j^l = \sigma \left(\sum_k w_{jk}^{(l-1)} a_k^{l-1} \right).$$

Learning with Gradient Descent

$$\text{Cost fn } C(w, b) = \frac{1}{2n} \sum_x \|y - \hat{a}^L\|^2.$$

$a^L \rightarrow$ activation of the last layer.

Initialize with w_0, b_0 .

At some step,

$$(w^{k+1}, b^{k+1}) = (w^k, b^k) - \eta \nabla C.$$

$\eta \rightarrow$ step size.

Talk about gradient, its defn & why
are we choosing the direction of gradient?

Stochastic Gradient Descent $\rightarrow$

$$C = \frac{1}{n} \sum C_x.$$

Need to calculate gradient for each data point.
This can be slow if n is very large.

Randomly pick out ' m '. Average over only m
of these. Referred to as mini-batch.

$$\frac{\sum_{j=1}^m \nabla C_{x_j}}{m} \approx \frac{\sum_x \nabla C_x}{n} = \nabla C, \quad (1.18)$$

where the second sum is over the entire set of training data. Summing sides we get

We can think of stochastic gradient descent as being like political polling: it's much easier to sample a small mini-batch than it is to apply gradient descent to the full batch, just as carrying out a poll is easier than running a full election. For example, if we have a training set of size $n=60,000$, as in MNIST, and choose a mini-batch size of (say) $m=10$, this means we'll get a factor of 6,000 speedup in estimating the gradient! Of course, the estimate won't be perfect – there will be statistical fluctuations – but it doesn't need to be perfect: all we really care about is moving in a general direction that will help decrease C , and that means we don't need an exact computation of the gradient. In practice, stochastic gradient descent is a commonly used and powerful technique for learning in neural networks, and it's the basis for most of the learning techniques we'll develop in this book.

Implementation

Training Set Split $\rightarrow$ 50,000 Training
10,000 Validation

Test Set $\rightarrow$ 10,000.

Randomly shuffle. Take mini-batches of size 10,
do a gradient descent step. Go over entire data.
This is one epoch.

30 hidden neurons $\rightarrow$ 1 hidden layer

784 $\rightarrow$ 30 $\rightarrow$ 10.

$\eta = 3.0 \rightarrow 95.42\%$.

100 hidden neurons $\rightarrow 96.59\%$.

these all run through the book, including how I chose the hyper-parameters above.

Exercise

- Try creating a network with just two layers – an input and an output layer, no hidden layer – with 784 and 10 neurons, respectively. Train the network using stochastic gradient descent. What classification accuracy can you achieve?

Remark: A well tuned SVM can get to 98.5%. Record till this book was written is 9,979 correctly classified images by NN. It seems like NN is considered state of the art.

Chapter 2

Back propagation

Want to calculate $\frac{\partial C}{\partial w}$ or $\frac{\partial C}{\partial b}$ for all weights & biases.

Notations: $w_{jk}^l \rightarrow$ weight from node k in layer $l-1$ to node j in layer l .

$a_j^l \rightarrow$ activations/values in node j in layer l .

$$\Rightarrow a_j^l = \sigma \left(\sum_k w_{jk}^l \cdot a_k^{l-1} \right)$$

In matrix notation,

$$a^l = \sigma \left(w^l \cdot a^{l-1} \right)$$

where σ acts co-ordinatewise.

$z_j^l \rightarrow$ values in node j , layer l before applying σ ;

$$z_j^l = \sum_k w_{jk}^l \cdot a_k^{l-1}.$$

$$\delta_j^L = \frac{\partial C}{\partial z_j^L}$$

Four Equations

$$1) \quad \delta_j^L = \frac{\partial C}{\partial a_j^L} \sigma'(z_j^L).$$

Pf: This is for the last layer L .

$$C(a_j^L(z_j^L))$$

$$\text{or, } z_j^L \rightarrow a_j^L \rightarrow C.$$

So, by chain rule

$$\frac{\partial C}{\partial z_j^L} = \frac{\partial C}{\partial a_j^L} \cdot \underbrace{\frac{\partial a_j^L}{\partial z_j^L}}_{\sigma'(z_j^L)}.$$

How to compute $\frac{\partial C}{\partial a_j^L}$?

E.g if $C = \frac{1}{2} \sum_j (y_j - a_j^L)^2$

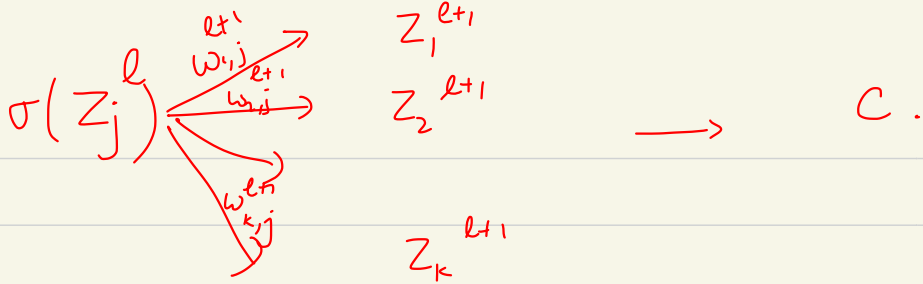
then $\frac{\partial C}{\partial a_j^L} = a_j^L - y_j$.

We would also need to have made one forward pass; so we would have calculated z_j^L (for the current values of the parameters).

$$2) \quad \delta^L = \left(W^{L+1} \right)^T \cdot \delta^{L+1} \odot \sigma'(z^L)$$

Entry wise product of two vectors.

Pf: Can compute δ^L if δ^{L+1} is computed.



$$z_j^l \rightarrow \sigma(z_j^l) \rightarrow \begin{pmatrix} z_1^{l+1} \\ \vdots \\ z_k^{l+1} \end{pmatrix} \rightarrow C.$$

$\parallel$
 a_j^l

$$S_o, \quad \frac{\partial C}{\partial z_j^l} = \left(\sum_k \frac{\partial C}{\partial z_k^{l+1}} \cdot \frac{\partial z_k^{l+1}}{\partial a_j^l} \right) \cdot \sigma'(z_j^l)$$

$\parallel$
 $\sum_k \frac{\partial C}{\partial a_j^l} \cdot w_{k,j}^{l+1}$

Multivariable Chain Rule

$$h(x) = x \rightarrow \begin{pmatrix} f_1(x) \\ \vdots \\ f_k(x) \end{pmatrix} \rightarrow g(f_1(x), \dots, f_k(x)).$$

$$h'(x) = \sum_j \frac{\partial g(f_1, \dots, f_k)}{\partial f_j} \cdot f_j'(x).$$

Proof for Chain Rule:

$$h(x+\varepsilon) - h(x)$$

$$= g(f_1(x+\varepsilon), \dots, f_k(x+\varepsilon)) - h(x)$$

$$g(f_1(x) + \varepsilon \cdot f_1'(x) + o(\varepsilon^2), \dots, f_k(x) + \varepsilon \cdot f_k'(x) + o(\varepsilon^2))$$

$$- h(x)$$

defn of
=
gradient

$$h(x) + (\varepsilon \cdot f_1'(x) + o(\varepsilon^2)) \cdot \frac{\partial g}{\partial f_1}(\dots)$$

$$+ \dots$$

$$- \cancel{h(x)}$$

$$= \varepsilon \cdot \left[\sum_j f_j'(x) \cdot \frac{\partial g}{\partial f_j}(f_1, \dots, f_j) \right] + o(\varepsilon^2).$$

So, we have

$$z_j^l = \left(\sum_k w_{kj}^{l+1} \cdot z_k^{l+1} \right) \cdot \sigma'(z_j^l)$$

Now we can write this in matrix form.

$$z^l = (W^T z^{l+1}) \odot \sigma'(z^l).$$

□.

$$3) \quad \frac{\partial c}{\partial b_j^l} = \delta_j^l.$$

$$b_j^l \rightarrow z_j^l \rightarrow c.$$

By chain rule,

$$\frac{\partial c}{\partial b_j^l} = \underbrace{\frac{\partial c}{\partial z_j^l}}_{\delta_j^l} \cdot \underbrace{\frac{\partial z_j^l}{\partial b_j^l}}_1.$$

□

$$4) \quad \frac{\partial C}{\partial w_{jk}^l} = a_k^{l-1} \cdot \delta_j^l.$$

(from k th node in $l-1$ to j th node in l)

$$w_{jk}^l \rightarrow z_j^l \rightarrow C.$$

$$\text{So, } \frac{\partial C}{\partial w_{jk}^l} = \underbrace{\frac{\partial C}{\partial z_j^l}}_{\parallel \delta_j^l} \cdot \underbrace{\frac{\partial z_j^l}{\partial w_{jk}^l}}_{\parallel a_k^{l-1}}.$$

- Some insights
- In what sense is this a fast algo? What is its comp. complexity?

Ch-1 - Intro

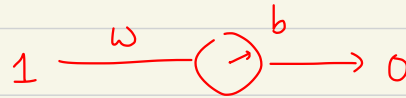
Ch2 $\rightarrow$ Backprop.

Ch3 $\rightarrow$ Derivatives; Cross-Entropy, Softmax,
 L_2 reg, L_1 reg,

Ch-6 $\rightarrow$ CNN architecture.

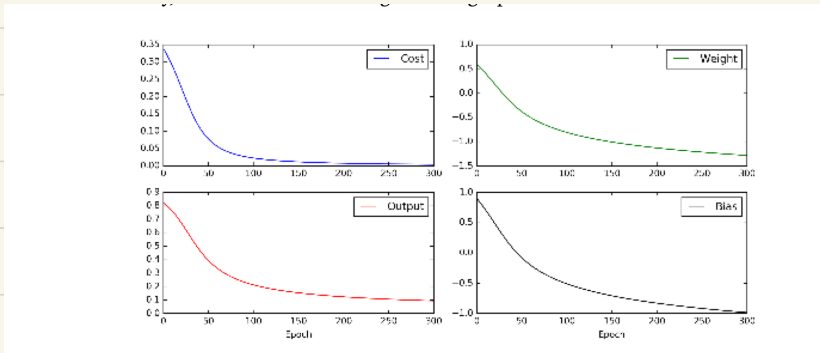
Chapter 3

Toy problem:

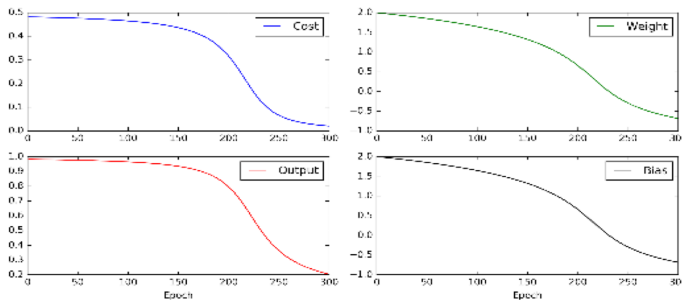


Initial weight = 0.6, bias = 0.9

Initial Output = 0.82, $\eta = 0.15$



Now, initial weight = bias = 2.0
Initial Output = 0.98, $\eta = 0.15$



'Slow
→ learning'
when
badly
wrong'

$$C = \frac{(y-a)^2}{2}, \quad (3.1)$$

where a is the neuron's output when the training input $x = 1$ is used, and $y = 0$ is the corresponding desired output. To write this more explicitly in terms of the weight and bias, recall that $a = \sigma(z)$, where $z = wx + b$. Using the chain rule to differentiate with respect to the weight and bias we get

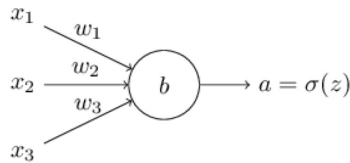
$$\frac{\partial C}{\partial w} = (a - y)\sigma'(z)x = a\sigma'(z) \quad (3.2)$$

$$\frac{\partial C}{\partial b} = (a - y)\sigma'(z) = a\sigma'(z), \quad (3.3)$$

where I have substituted $x = 1$ and $y = 0$. To understand the behavior of these expressions, let's look more closely at the $\sigma'(z)$ term on the right-hand side. Recall the shape of the σ function:

When z is very close to 1 (or 0); $\sigma'(z)$ becomes very small.

Cross Entropy Loss



The output from the neuron is, of course, $a = \sigma(z)$, where $z = \sum_j w_j b_j + b$ is the weighted sum of the inputs. We define the cross-entropy cost function for this neuron by

$$C = -\frac{1}{n} \sum_x [y \ln a + (1 - y) \ln(1 - a)], \quad (3.4)$$

where n is the total number of items of training data, the sum is over all training inputs, x , and y is the corresponding desired output.

It's not obvious that the expression (3.4) fixes the learning slowdown problem. In fact

If we think of output $a(x) = \text{Rob}(y(x) = 1)$; y binary scalar.

then likelihood $\propto \prod_x a(x)^{y(x)} (1 - a(x))^{1 - y(x)}$

$$\log \text{likelihood} \propto \sum y(x) \log a(x) + (1 - y(x)) \log (1 - a(x))$$

$C \geq 0, \quad = 0 \text{ iff } a = y \quad \forall x.$

$$\frac{\partial C}{\partial w_j} = -\frac{1}{n} \sum_x \left(\frac{y}{\sigma(z)} - \frac{1-y}{1-\sigma(z)} \right) \frac{\partial \sigma}{\partial w_j} = -\frac{1}{n} \sum_x \left(\frac{y}{\sigma(z)} - \frac{1-y}{1-\sigma(z)} \right) \sigma'(z) x_j. \quad (3.5)$$

Putting everything over a common denominator and simplifying this becomes:

$$\frac{\partial C}{\partial w_j} = \frac{1}{n} \sum_x \frac{\sigma'(z) x_j}{\sigma(z)(1-\sigma(z))} (\sigma(z) - y). \quad (3.6)$$

Using the definition of the sigmoid function, $\sigma(z) = 1/(1 + e^{-z})$, and a little algebra we can show that $\sigma'(z) = \sigma(z)(1 - \sigma(z))$. I'll ask you to verify this in an exercise below, but for now let's accept it as given. We see that the $\sigma'(z)$ and $\sigma(z)(1 - \sigma(z))$ terms cancel in the equation just above, and it simplifies to become:

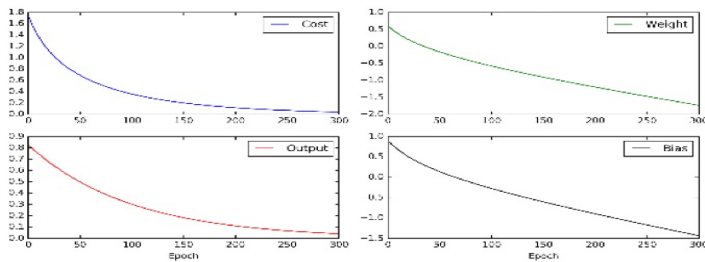
$$\frac{\partial C}{\partial w_j} = \frac{1}{n} \sum_x x_j (\sigma(z) - y). \quad (3.7)$$

There is no $\sigma'(z)$ term; proportional to $\sigma(z) - y$, which is what we want. Learn faster when more badly wrong.

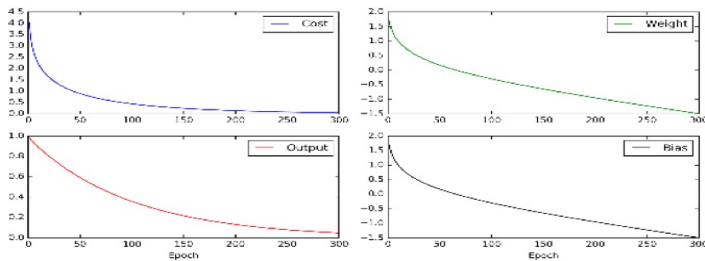
$$\frac{\partial C}{\partial b} = \frac{1}{n} \sum_x (\sigma(z) - y).$$

- Verify that $\sigma'(z) = \sigma(z)(1 - \sigma(z))$

Let's return to the toy example we played with earlier, and explore what happens when we use the cross-entropy instead of the quadratic cost. To re-orient ourselves, we'll begin with the case where the quadratic cost did just fine, with starting weight 0.6 and starting bias 0.9:



Unsurprisingly, the neuron learns perfectly well in this instance, just as it did earlier. And now let's look at the case where our neuron got stuck before, with the weight and bias both starting at 2.0:



For a general neural net with multiple output

neurons; define cross-entropy loss as
(for one data pt.; y is a vector)

$$\sum_j [y_j \ln a_j^L + (1 - y_j) \ln(1 - a_j^L)].$$

Remark: This is why cross-entropy loss better choice than sq. loss when using sigmoid activation.

Use this expression to show that the partial derivative with respect to the weights in the output layer is given by

$$\frac{\partial C}{\partial w_{jk}^L} = \frac{1}{n} \sum_x a_k^{L-1} (a_j^L - y_j). \quad (3.13)$$

↓

Multiple Layers; Derivative
w.r.t final layer.

Experiments

1 hidden layer, 30 hidden neurons, mini-batch size = 10, $\eta = 0.5$, 30 epochs.

We get 95.49%, similar to 95.42% for quadratic loss.

100 hidden neurons.

We get 96.82%, bettering 96.59% for quadratic loss.

Of course; these are not comprehensive judgements. One needs to search over all hyperparameters.

Another Loss Fn to avoid learning slowdown

In the last layer, we form

$$z_j^L = \sum_k w_{jk}^L a_k^{L-1} + b_j^L.$$

We don't apply sigmoid. We apply softmax.

$$a_j^L = \frac{e^{z_j^L}}{\sum e^{z_j^L}}.$$

Log Likelihood Cost Fn $\rightarrow$

For datapoint with value y_j ;
we use the loss $-\log a_j^L$.

$$\frac{\partial C}{\partial b_j^L} = a_j^L - y_j \quad (3.27)$$

$$\frac{\partial C}{\partial w_{jk}^L} = a_k^{L-1}(a_j^L - y_j) \quad (3.28)$$

↓

With a little algebra; can show the above.

Regularization (To reduce overfitting)

$$C = -\frac{1}{n} \sum_{x_j} [y_j \ln a_j^L + (1 - y_j) \ln(1 - a_j^L)] + \frac{\lambda}{2n} \sum_w w^2. \quad (3.31)$$

30 hidden neurons, mini-batch size = 10,
learning rate $\eta = 0.5$, $\lambda = 5.0$

Classification Accuracy : Up from 95.49%
to 96.49%..

100 hidden neurons, 60 epochs, $\eta = 0.1$, $\lambda = 5.0$,
98.04% accuracy.

Initialization: Instead of initializing with $N(0,1)$
for the weights, initialize with $N(0, \frac{1}{\sqrt{\# \text{input neurons}}})$

So that the weighted comb is not too large in magnitude.

Hyperparameters $\rightarrow$ learning rate $\rightarrow \eta$

Regularization $\rightarrow \lambda$

epochs to run $\rightarrow$

'One can easily choose bad hyperparameters'.

Nielsen 'tunes' on Validation Data; then checks finally on test data.

Chapter 6 : (CNN).

Chapter 6 : Convolutional Neural Networks

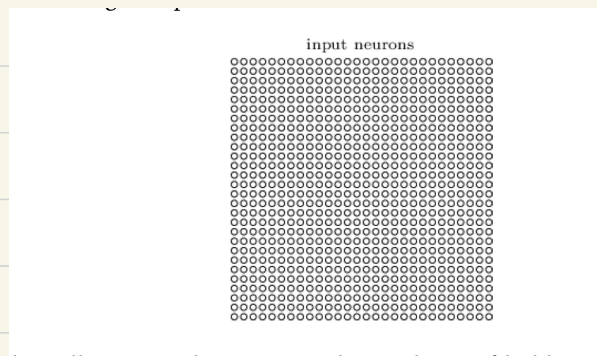
Fully connected NN did a pretty good job on MNIST. About 98% accuracy.

Fully connected seems a bit strange. Every pixel is treated equally. No spatial structure used.

CNN uses 3 basic ideas.

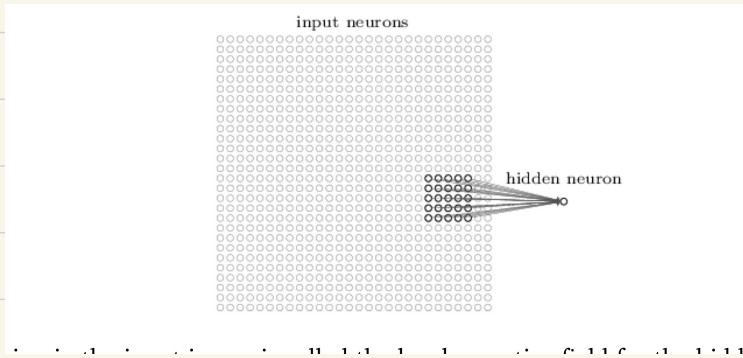
- Local Receptive Fields
- Shared Weights
- Pooling.

a) Local Receptive Fields

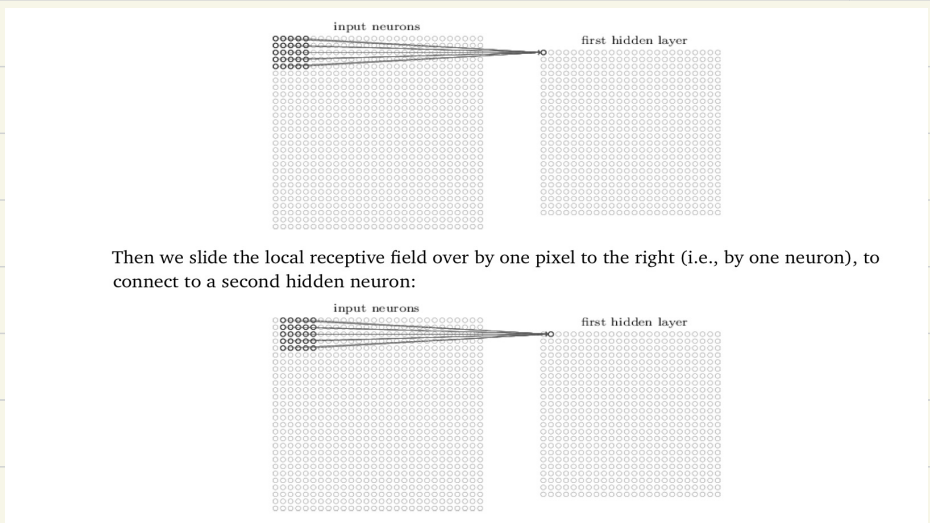


→ Think of input as 28×28 matrix.

Each neuron in first hidden layer connected to a small patch, say 5×5 of input neurons.



As usual, each connection has a weight and each neuron in the hidden layer has a bias.



Then we slide the local receptive field over by one pixel to the right (i.e., by one neuron), to connect to a second hidden neuron:

If we have a 28×28 image; since we have 5×5 local fields; we will have 24×24 first hidden layer.

Here, stride length is 1; people experiment with different strides.

b) Shared Weights & Biases

We use same weights & bias for each of the (24×24) neurons in the first hidden layer.

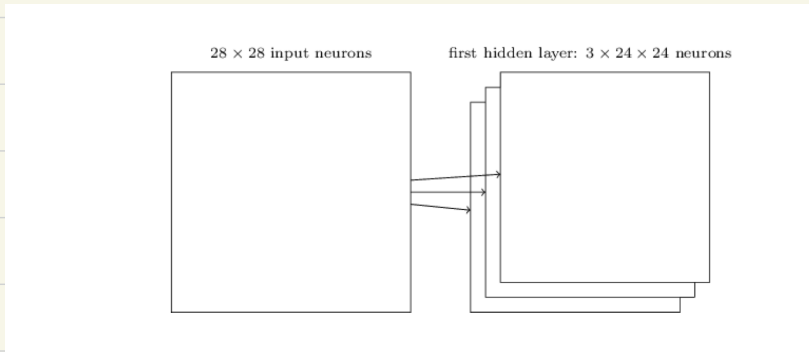
For j, k the neuron; output is

$$\sigma \left(b + \sum_{l=0}^4 \sum_{m=0}^4 w_{l,m} a_{j+l, k+m} \right).$$

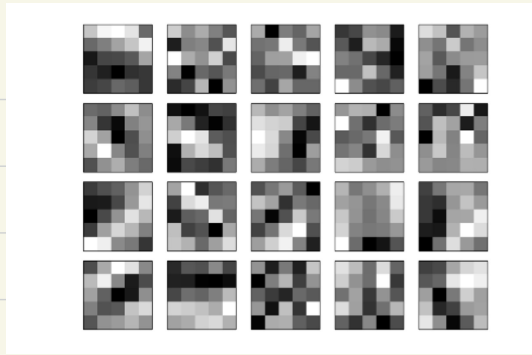
$a_{x,y} \rightarrow$ input activation at pixel x,y .

All neurons in hidden layer 1 detect some 'feature'; say a vertical edge. So we go through the entire image to detect the 'feature'. People say this is why CNN's are 'translation invariant'; image of a cat rotated is still a cat.

Shared Weight + Shared Bias $\rightarrow$ 'Filter'



We will use multiple 'feature' detectors; one may not be enough. One of the early CNN's, LeNET-5 used 6 feature maps to recognize MNIST digits.



20 learned
→ feature
maps.
Black → Higher
Pixel Value.

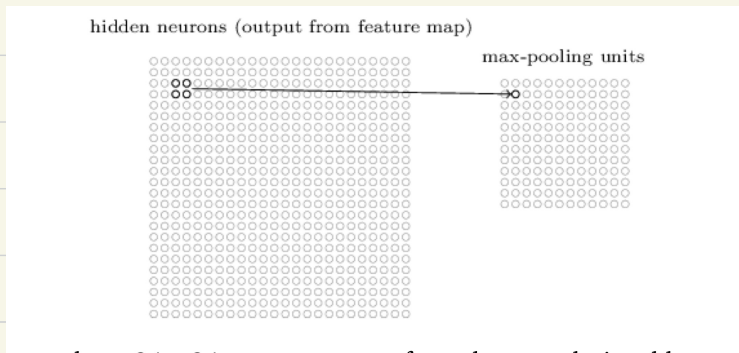
parameters per feature map → 5×5 shared
weights + one bias = 26. If we have
20 maps then $20 \times 26 = 520$.

Compare to fully connected NN; with 30 hidden
neurons, then $784 \times 30 + 30$.

c) Pooling

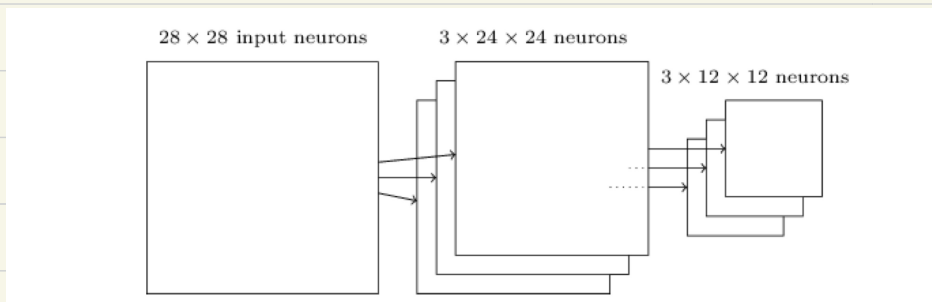
Pooling layer used immediately after convolutional
layer. Summarizes a region (say 2×2) of conv.
layer.

One example is max pooling. A pooling neuron outputs maximum of 2×2 region.



If we have 28×28 conv. layer; then after pooling layer we have 12×12 .

We apply pooling to each feature map separately.

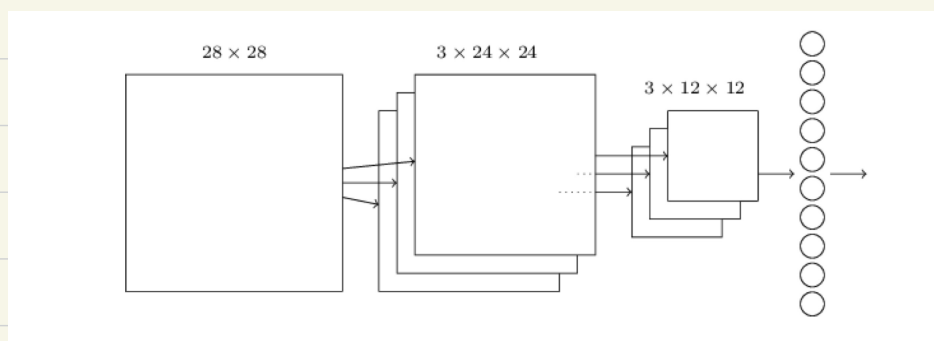


What is the intuition behind pooling?

"A way for the network to code whether a given feature is present or not; throwing away some positional info".

Remark: People also do L_2 pooling.

Putting it all together:



Input $\rightarrow 28 \times 28$ Neurons (MNIST)

Conv. (Hidden) Layer $\rightarrow 3$ feature maps with 5×5 local fields.

Pooling (Hidden) Layer $\rightarrow 3$ 2×2 max pooling layer.

Final (Fully Connected) Layer $\rightarrow 10$ output neurons each connected to all neurons in previous layer.

We use backpropagation & SGD as well.

Exercise: How is backprop. modified?

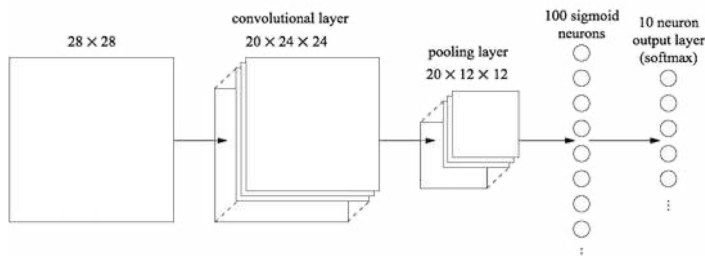
In Practice

Baseline $\rightarrow$ Shallow Architecture; Contains 100 hidden neurons, 60 epochs, learning rate $\eta = 0.1$, mini-batch size = 10, no regularization.

Best classification accuracy $\rightarrow$ 97.8%.

'Evaluated on test data at the epoch where we get best accuracy on validation data.'

CNN $\rightarrow$ Now we use this.



We get 98.78% $\rightarrow$ Considerable Improvement.

Exercise

- What classification accuracy do you get if you omit the fully-connected layer, and just use the convolutional-pooling layer and softmax layer? Does the inclusion of the fully-connected layer help?

Can we improve on the 98.78 percent classification accuracy?

Insert a second convolution + pooling layer.

We get 99.07%.

Here, after first pooling, we have $20 \times 12 \times 12$ images; Each neuron in the second conv. layer learns from all $20 \times 5 \times 5$ input neurons with shared weights for each input image.

RELU Activation

Change activation to RELU.

Learning rate $\eta = 0.03$, 60 epochs, L_2 regularization $\lambda = 0.1$.

Classification Accuracy $\rightarrow 99.23\%$.

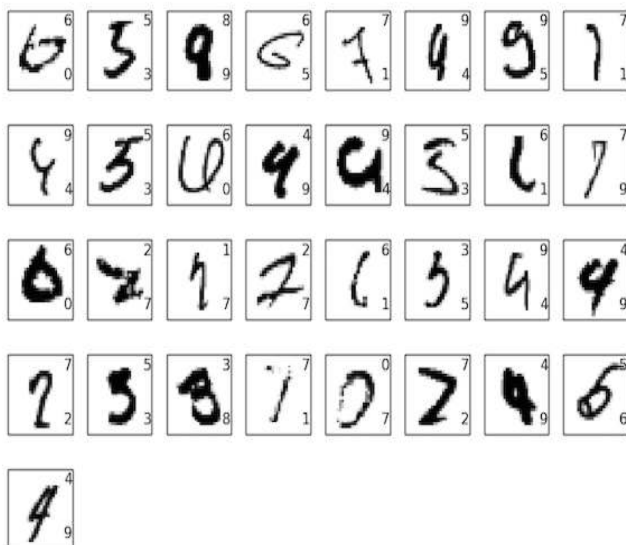
People have found there is real gain in using RELU over sigmoid or tanh.

Qn: Why is ^{non-}differentiability not a problem?

Ensemble

An easy way to improve performance is to create several ⁽⁵⁾ NN; get them to vote to determine classification.

In Nielsen's book $\rightarrow$ final accuracy $\rightarrow 99.67\%$.



Top right →
Correct

Bottom right
→

NN answer

Some examples of falsely classified digits.